

or meetings with customers, please include a briefing doc and information about any current executive sponsors.

SECTION: eba/vp-scheduling.md

When scheduling for VPs or Directors at GitLab, please check the scheduling preferences below.

Scheduling for Rob Allen, VP, Talent Acquisition

- Please schedule all meetings through the People Function's EBA, Julia Nishioki
- Keep meetings to 25 or 50 minutes unless otherwise specified
- Direct Report 1:1's occur weekly
- Skip level 1:1's occur every other month
- Calendar color coding: Confirmed meetings- Dark green, Manager meetings- light green, Important meetings NOT to be moved are in red, Personal meetings- Orange
- Do not schedule over DNB: Focus Time blocks without consulting the EBA of the People Group.
- Scheduling should ideally be kept between 8:00 am and 4:00 pm PT

Scheduling for Sherida McMullan VP, Diversity, Inclusion and Belonging

- Please schedule all meetings through the People Function's EBA, Julia Nishioki
- Keep meetings to 25 or 50 minutes unless otherwise specified
- Direct Report 1:1's occur weekly
- Calendar color coding: Confirmed meetings- Dark green, Manager meetings- light green, Important meetings NOT to be moved are in red, Personal meetings- Brown or Purple
- Do not schedule over DNB: Focus Time blocks without consulting the EBA of the People Group.
- Scheduling should ideally be kept between 8:00 am and 4:00 pm PT

Scheduling for Sales VPs

David Sakamoto, Mike Pyle, Ryan O'Neill, & Nima Badiy

- Please reach out to Victoria Reinhardt (Sr. EBA) to schedule or reschedule any meetings for Ryan O'Neill and Mike Pyle
- Please reach out to Becky Piccone (Sr. EBA) to schedule or reschedule any meetings for David Sakamoto and Nima Badiy
- Don't schedule over DNB or any CRO calls without reaching out for approval from VP and EBA
- For interviews: CES should tag EBA on Greenhouse to review times submitted by the candidate, or to provide times for the VP
- If you add any meetings to calendar directly, tick the box "Attendees can modify" so EBA can change if necessary
- External meeting requests (with those outside of GitLab): email EBA and VP and loop them in with external parties. EBA will then schedule directly with external parties.
- Internal meeting requests (with other GitLab Team Members)
 - Please slack EBA in sales-vp channel
 - If the meeting is confidential (ex: interviews), please slack EBA directly
- Please include the following in your slack/email
 - Must have/optional attendees

- Meeting type: internal prep, client facing/customer, prospective customer, etc
- Urgency/ Desired timeframe: in the next two days, in the next week or two, etc
- Duration: 25 mins, 50 mins, etc.
- Subject of the meeting
- Provide context: include agenda to be covered. Share google doc if available to be included in invite or link relevant issues, slides, etc.

Scheduling for Finance VPs and Sr. Leadership

James Shen, VP of Finance - Nabitha Rao, VP of EntApps - Kelsey Turcotte, VP of Investor Relations - Jim Gladen, VP of Tax - Julie Malloy, VP Accounting Operations - Simon Mundy, VP Controller

- Please reach out to Anita Mall (Sr. EBA) to schedule or reschedule any meetings with James Shen, Nabitha Rao, or Kelsey Turcotte
- Please reach out to Sarah Hobbs (EBA) to schedule or reschedule any meetings with Jim Gladen, Julie Malloy or Simon Mundy
- Don't schedule over DNB blocks
- For interviews: CES should tag EBA on Greenhouse to review times submitted by the candidate, or to provide times for the VP
- If you add any meetings to calendar directly, tick the box "Attendees can modify" so EBA can change if necessary
- Working blocks on the calendar are OK and can be moved to prioritize other meeting requests, team members should check in with the EBA to request a meeting using the meeting request requirements as a guideline
- External meeting requests (with those outside of GitLab): email EBA and VP / Sr. Leadership and loop them in with external parties. EBA will then schedule directly with external parties.
- For internal meeting requesting please tag the EBA on eba-team slack channel, include the following in your slack/email:
 - Must have/optional attendees
 - Urgency/ Desired timeframe: in the next two days, in the next week or two, etc
 - Duration: 25 mins, 50 mins, etc.
 - Subject of the meeting
 - Provide context: include agenda to be covered, share google doc if available, include a link to any relevant issues, slides, etc.

Scheduling for Product Leadership

Mike Flouton, VP of Product Management - Justin Farris, VP Product Management - Angela Pesta, VP UX - Hillary Benson, Sr Director of Product Management - Steve Evangelista, Sr Director of Product Management

- Please reach out to Jennifer Garcia (EBA) to schedule or reschedule any meetings for Justin Farris, and Angela Pesta
- Please reach out to Lindsey Beyl (EBA) to schedule or reschedule any meetings for Mike Flouton, Hillary Benson and Steve Evangelista
- Don't schedule over DNB and Work Block's without reaching out to Jennifer and Lindsey (EBA)
- For internal and external (with those outside of GitLab) meeting requesting please contact Product EBAs in eba-team, including the following in your slack:
 - Must have/optional attendees

- Urgency/ Desired timeframe: in the next two days, in the next week or two, etc
- Duration: 25 mins, 50 mins, etc.
- Subject of the meeting
- Provide context: include agenda to be covered, share google doc if available, include a link to any relevant issues, slides, etc.
 - Please ensure the leader and EBA have access to view the materials
 - For customer meetings please create an issue request using this template and tag Jennifer Garcia (Product EBA) and Lindsey Beyl (Product EBA) for review and availability
 - If you add any meetings to calendar directly, tick the box "Attendees can modify" so EBA can change if necessary

Scheduling for Engineering Leadership

Audrey Van Belleghem, Chief of Staff - Michelle Gill, Director of Devops Engineering - Tim Zallmann, VP AI Engineering - Maw Wildpaner, VP of Sec & Monetization - Marin Jankovski, VP of Infrastructure Platforms - Johnny Scarborough Jr., VP of Support

- Please reach out to Liki Simonot (EBA) to schedule or reschedule any meetings for Tim Zallmann, Marin Jankovski, or Maw Wildpaner
- Please reach out to Vange Ingan (EBA) to schedule or reschedule any meetings for Audrey Van Belleghem
- Please reach out to Kyra Wright (EBA) to schedule or reschedule any meetings for Michelle Gill, or Johnny Scarborough Jr.

SECTION: engineering/_index.md

The GitLab Product team looks ahead for expanding the platform ·What" (customer needs) and "Why· (business strategy) and Engineering determines the ·How" (technical implementation) and "When· (scheduling) of the platform releases. The content on this page talks about how we do engineering at GitLab.

Engineering Direction

GitLab has a Three-Year Strategy, and we're excited to see every member of the Engineering division contribute to achieving it. Whether you're creating something new or improving something that already exists, we want you to feel empowered to bring your best ideas for influencing the product direction through improved scalability, usability, resilience, and system architectures. And when you feel like you need to expand your knowledge in a particular area, know that you're supported in having the resources to learn and improve your skills.

Our focus is to make sure that GitLab is enterprise grade in all its abilities and to support the AI efforts required to successfully launch AI features to General Availability.

Making sure that GitLab is enterprise grade involves several teams collaborating on improving our disaster recovery and support offerings through ongoing work with GitLab Dedicated and Cells infrastructure. Our goal here is improved availability and service recovery.

Engineering Culture

Engineering culture at GitLab encompasses the processes, workflows, principles and priorities that all stem from our GitLab Values.

All these things continuously strengthen our engineering craftsmanship and allow engineers to achieve engineering excellence, while growing and having a significant, positive impact on the product, people, and the company as a whole. Our engineering culture is primarily being carried and evolves through knowledge sharing and collaboration. Everyone can be part of this process because at GitLab everyone can contribute.

Engineering Excellence

Engineering excellence can be defined as an intrinsic motivation to improve engineering efficiency, software quality, and deliver better results while building software products. Engineering excellence is being fueled by a strong engineering culture combined with a mission: to build better software that allows everyone to contribute.

Engineering Initiatives

Engineering is the primary advocate for the performance, availability, and security of the GitLab project. Product Management prioritizes 60% of engineering time, so everyone in the engineering function should participate in the Product Management prioritization process to ensure that our project stays ahead in these areas. Engineering prioritizes 40% of time on initiatives that improve the product, underlying platform, and foundational technologies we use.

Work in the 40% time budget should be coordinated and prioritized by the Engineering Manager of a team. Use the label Engineering Time for issues and MRs that are done as part of it so we can follow the work and the results across the engineering division.

- Contributing to broad engineering initiatives and participating in working group-related tasks.
- Review fixes from our support team. These merge requests are tagged with the Support Team Contributions label. You can filter on open MRs.
- Working on high priority issues as a result of issue triaging. This is our commitment to the community and we need to include some capacity to review MRs or work on defects raised by the community.
- Improvements to the performance, stability and scalability of a feature or dependency including underlying infrastructure. Again, the Product team should be involved in the definition of these issues but Engineering may lead here by planning, prioritizing, and coordinating the recommended improvements.
- Improvements and upgrades to our toolchain in order to boost efficiency.
- Codebase improvements: Removing technical debt, updating or replacing outdated dependencies, and enhancing logging and monitoring capabilities.
- Constructing Proof-of-Concept models for thorough exploration of new technologies, enhancements and new possibilities.
- Work on improvements and feature enhancements to the product, in the sense of internal community contributions, that would increase our internal engineering productivity by focusing on ready-to-go items that are currently assigned a low priority in the backlog.

Engineering Innovation

Engineering Innovation is a new process geared toward individual or small-team collaboration that encourages engineers to explore new ideas and Proof-of-Concepts. These projects are typically lean, time-boxed, iterative, and designed to validate whether an idea has the potential to evolve into a viable experimental feature or product. See the Innovation at GitLab Guide.

Technical Roadmaps

Some of the above examples for the 40% time budget can help in forming a long-term technical roadmap for your group, and determine how best to prioritize your technical work to support overall business goals. In addition to the examples above:

- Ask yourself these questions
 - What are your most frequent sources of delays? (Could be long-standing tech debt you have to work past while developing, could be lack of reviewers for your domain, could be external to your team like with pipeline duration)
 - Do you have any consistently similar bugs or security issues that come in due to a certain area?
 - Has your team been talking about potentially refactoring any areas?
 - Is your team struggling with certain processes?
 - Have you had recent incidents that allude to a larger problem?
 - Are you getting frequent requests for help in some area?
 - Is your team frequently missing their deliverable commitments? What would help?
 - Does your area have performance (slow endpoints, inconsistent responses, intermittent errors) or scalability (the feature or area as-is will not scale) concerns?
 - Where do you see the biggest instability? Have you talked to operations and support about feedback for your area?
 - Do you have application or rate limits in the right places?
 - Have you burned down your security, corrective action, and infradev issues?
 - Is your error budget green?
 - Have your feature flags been removed from the codebase yet?
 - Do you have adequate unit test, integration test and E2E coverage?
 - Do you have adequate documentation for your features?
 - Do you have adequate telemetry, logging, monitoring of your features?
 - Do you have adequate error handling and error codes that allows fast and easy diagnostics?
- Gather data like this
 - Master:Broken issues
 - ~"severity::1" and ~"severity::2" bugs
 - Missed-Slo issues
 - Flaky test issues
 - ~"type::maintenance" issues
- Think about the future state of your product
 - Where do you want your product to be this time next year?
 - What are the technical requirements to achieve that?
 - What are technical topics that would benefit from research/POCs?
 - What would make it easier for you to achieve that if it was no longer a factor?
 - What would be the performance and/or business impact once you address these issues?
 - How would you evolve your team processes to regularly review your technical roadmap?

Technical roadmap process

Engineering Managers (EMs) are responsible for collaboratively developing their team's technical roadmap backlog. All items should be documented as epics and issues using the "Technical Roadmap" label.

Global initiatives will be defined and must be incorporated into each group's roadmap and prioritization (e.g., allocating 40% of front-end capacity for Vue upgrade, completing all Cells issues for a specific area by milestone XYZ).

Prioritization of items should align with:

1. General business goals
2. Engineering vision
3. Team capacity and expertise

Planning Guidelines:

- Allocate 40% of the overall time budget for technical roadmap items in the normal milestone planning process.
- Use the "Technical roadmap" label for all related issues to facilitate tracking and coordination.

Key Steps:

1. Identify and document technical debt and improvement opportunities
2. Assess impact and effort for each item
3. Prioritize based on business value and strategic alignment
4. Integrate with existing iteration/milestone planning
5. Regularly review and adjust the roadmap

This process ensures a balanced approach between feature development and technical improvements, promoting long-term sustainability and efficiency of the engineering organization.

Community Contributions

We have a 3-year goal of reaching 1,000 monthly contributors as a way to mature new stages, add customer-desired features that aren't on our roadmap, and even translate our product into multiple languages.

Diversity

Diverse teams perform better. They provide a sense of belonging that leads to higher levels of trust, better decision making, and a larger talent pool. They also focus more on facts, process facts more carefully, and are more innovative. By hiring globally and increasing the numbers of women and under represented groups (URGs) in the Engineering division, we're helping everyone bring their best selves to work.

Growing our team

Strategic hiring is a top priority, and we're excited to continue hiring people who are passionate about our product and have the skills to make it the best DevSecOps tool in the market. Our current focus areas include reducing the amount of time between offer and start dates and hiring a diverse team (see above). We're also implementing industry-standard approaches like structured, behavioral, and situational interviewing to help ensure a consistent interview process that helps to identify the best candidate for every role. We're excited to have a recruiting org to partner with as we balance the time that managers spend recruiting against the time they spend investing in their current team members.

Expand customer focus through depth and stability

As expected, a large part of our focus is on improving our product.

For Enterprise customers, we're refining our product to meet the levels of security and reliability that customers rightfully demand from SaaS platforms (SaaS Reliability). We're also providing more robust utilization metrics to help them discover features relevant to their own DevOps transformations (Usage Reporting) and offering the ability to purchase and manage licenses without spending time contacting Sales or Support (E-Commerce and Cloud Licensing). Lastly, in response to Enterprise customer requests, we're adding features to support Suggested Reviewers, better portfolio management through Work Items, and Audit Events that provide additional visibility into user passive actions.

For Free Users, we're becoming more efficient with our open core offering, so that we can continue to support and give back to students, startups, educational institutions, open source projects, GitLab contributors, and nonprofits.

For Federal Agencies, we're obtaining FedRAMP certification to strengthen confidence in the security standards required on our SaaS offering. This is a mandated prerequisite for United States federal agencies to use our product.

For Hosted Customers, we're supporting feature parity between Self-Managed and GitLab Hosted environments through the Workspace initiative. We're also launching GitLab Dedicated for customers who want the flexibility of cloud with the security and performance of a single-tenant environment.

For customers using CI/CD, we're expanding the available types of Runners to include macOS, Linux/Docker, and Windows, and we're autoscaling build agents.

Taking time off

{{% note %}}

This process is expected for PTO that is five consecutive days or more, inclusive of adjacent public holidays (excluding weekend days). For PTO that is fewer than five consecutive days, including the cases where there are multiple PTO blocks with fewer than 5 consecutive days and a few working days in-between, a coverage issue is not required but a coverage issue can be filed for PTO of any length, especially if it'd be helpful to balance team continuity and individual flexibility.

{{% /note %}}

In order to ensure business continuity, and deliver on commitments; the Engineering Division is

adopting a PTO Coverage Issue Process. Processes like this are already formalized in GitLab (e.g. PM Coverage Issue) and some team's within Engineering have practiced this regularly at the Management+ level. This allows us to continue to support team member well-being through time away without negatively impacting the rest of the team.

A PTO Coverage issue is required for job grades 8 and up. For job grades 7 and below a PTO Coverage issue is recommended as there is value in going through the process of creating the PTO Coverage issue even if there are minimal items to include (for all levels) in that it forces you to think about what you have on your plate and what impact your PTO will have on those items. So whether the result is that the work waits or there is someone designated as a replacement DRI, it makes the decision explicit and documented.

Once planning for a milestone has been completed (see Monday, 5 days before the milestone begins) PTO for periods longer than 5 consecutive days, inclusive of adjacent public holidays (excluding weekend days), cannot be requested. This is to prevent disrupting plans for that milestone. There are exceptions to this, but all need to be discussed with your manager. Examples include:

- urgent scenarios
- a team/individual hits targets earlier in the milestone (we measure impact, not activity)
- a strong need for a team member to take PTO of this length

These issues will help inform teams as they plan their milestones to ensure the work teams are committed to can be achieved with the staff available, or if there will be a lack of staff to achieve those commitments, to work with team members to see what can be done to achieve the results for our customers.

The process below helps to clarify and expand upon the Flexible PTO Policy by making the coordination with the team members manager explicit.

1. Creating an Engineering coverage issue

You should use this issue template to define handshake responsibilities. For extended leave, it is important to find one or more Directly Responsible Individuals (DRIs) that will be able to make decisions while you are away. This may be your manager, another engineer, or maybe the Product Manager for your team. The coverage issue should contain all the necessary information for the DRIs to make good decisions in your absence, so please make sure to include as much detail as needed. The coverage issue should highlight work impact estimates, mitigations identified, and coverage alternatives.

If additional context needs to be shared to provide color to the coverage issue, you can consider a specific handover meeting to cover further details.

It is recommended to work with your manager and other stakeholders when considering cross-functional teammate capacity for a coverage task assignment. For example, while it's optimal for PM, EM, and PDs to assist in covering for each other given their shared knowledge of their product area including customers and users, PM teammates may or may not have the bandwidth or expertise to take on covering engineering specific responsibilities. Alternatively, it may be better for the manager of the engineer or another engineer in the same stage to aid in coverage. Plan to have the necessary conversations across teams and managers.

2. Sharing your Engineering coverage issue with your manager

Once you've filed your engineering coverage issue, share this with your manager prior to milestone planning so they can review and approve. Check the latest guidance in our PTO policy on how much notice is required.

Consider whether any new commitments would be affected by your planned PTO. If a team member falls behind on something, they will need to make sure they have a coverage plan in place to ensure success of their commitments.

3. Manager reviews coverage issue

Once the team member has shared their coverage issue with their manager, the manager will review the coverage issue and validate assumptions with stakeholders or impacted project DRIs as needed.

The manager will make a decision on approval or discuss different arrangements or other contingency plans. Once the manager ticks their box on the coverage issue approving the leave, enter the time off into Workday.

4. Communicate your time off

After team members' coverage issue is approved, team members will communicate their time off and enter the PTO into Deel/Workday including a link to their coverage issue. Team members will share their coverage issue with their relevant colleagues via Slack channels, GitLab status, etc. ahead of the milestone planning.

5. Take your time off

Please disconnect and take the time off that you need!

6. Returning from Time Off

Returning from time off can be overwhelming and daunting. You should work with your DRIs to understand what has changed during your absence and what the current priorities are. Also, communicate transparently that your response time may be slower because you are catching up. Here are some additional tips on how to return back to work after time off.

Engineering Departments

There are five departments within the Engineering Division:

- DevOps Engineering Department
- AI Engineering Department
- Expansion Development Department
- Infrastructure Platforms
- Support Engineering Department

Other Related Pages

- CTO Leadership Team
- Communication
- Database Engineering
- Development Principles
- Engineering Automation
- Engineering Metrics
- Engineering OKRs
- Engineering READMEs
- Frequently Used Projects
- GitLab Innovation Program, managed by the GitLab Legal Team
- Hiring
- Mentorship
- Pajamas Design System
- R&D Tax Credit Applications

Workflows

- Developer onboarding
- Engineering Demo Process
- Engineering Workflow
 - Code Review
 - Security Issues
 - Architecture Design
- GitLab Repositories
- Issue Triage Policies
- Contributing to Go projects
- Wider Community Merge Request Triage Policies
- Root Cause Analysis
- Unplanned Critical Patch releases
- Incident Management

GitLab in Production

- Workflow Diagram
- Error Budgets
- Performance of GitLab
- Monitoring of GitLab.com
- Production Readiness Guide

People Management

- Engineering Career Development
- Engineering Career Mobility Principles
- Emerging Talent @ GitLab
- Engineering Secondments
- Engineering Management
- Starting New Teams

Cross-Functional Prioritization

See the Cross-Functional Prioritization page for more information.

SaaS Availability Weekly Standup

To maintain high availability, Engineering runs a weekly SaaS Availability standup to:

- Review high severity (S1/S2) public facing incidents
- Review important SaaS metrics
- Track progress of Corrective Actions
- Track progress of Feature Change Locks

Infrastructure Items

Each week the Infrastructure team reports on incidents and key metrics. Updating these items at the top of the

Engineering Allocation Meeting Agenda

is the responsibility of the Engineering Manager for the General Squad in Reliability.

1. Incident Review

- Include any S1 incidents that have occurred since the previous meeting.
- Include any incidents that required a status page update.

1. SaaS Metrics Review

1. Include screenshots of the following graphs in the agenda.

- Alert Volume Review,meta:(alias:!n,disabled:!f,index:b35d9ca0-6c67-11eb-968b-c18082d502f4,key:type,negate:!f,params:(query:pagerduty),type:phrase),query:(matchphrase:(type: pagerduty))),('\$state':(store:globalState),meta:(alias:!n,disabled:!f,index:b35d9ca0-6c67-11eb-968b-c18082d502f4,key:status,negate:!f,params:(query:acknowledged),type:phrase),query:(matchphrase:(status:acknowledged))),refreshInterval:(pause:!t,value:0),time:(from:now-2M,to:now-1h)))

- Corrective Actions

Development Items

For the core and expansion development departments, updates on current status of:

1. Error budgets

1. Reliability issues (infradev)

1. Security issues

Groups under Feature Change Locks should update progress synchronously or asynchronously in the weekly agenda.

The intention of this meeting is to communicate progress and to evaluate and prioritise escalations from infrastructure.

Feature Change Locks progress reports should appear in the following format in the weekly agenda:

FCL xxxx - [team name]

- FCL planning issue: <issue link>

- Incident Issue: <issue link>
- Incident Review Issue: <issue link>
- Incident Timeline: <link to Timeline tab of the Incident issue>
 - e.g. time to detection, time to initiate/complete rollback (as applicable), time to mitigation
- Cause of Incident
- Mitigation
- Status of Planned/completed work associated with FCL

{{% include "includes/feature-change-locks.md" %}}

Engineering Performance Indicator process

The Product Analytics team is responsible for maintaining Engineering Performance Indicators. Work regarding KPI / RPI is tracked using the Product Analytics task intake tracker.

Manual verification

We manually verify that our code works as expected.

Automated test coverage is essential, but manual verification provides a higher level of confidence that features behave as intended and bugs are fixed.

We manually verify issues when they are in the workflow::verification state.

Generally, after you have manually verified something, you can close the associated issue. See the Product Development Flow to learn more about this issue state.

We manually verify in the staging environment whenever possible.

In certain cases we may need to manually verify in the production environment.

If you need to test features that are built for GitLab Ultimate then you can get added to the issue-reproduce group on production and staging environments by asking in the development Slack channel. These groups are on an Ultimate plan.

Critical Customer Escalations

We follow the below process when existing critical customer escalations requires immediate scheduling of bug fixes or development effort.

Requirements for critical escalation

- Customer is in critical escalation state
- The issues escalated have critical business impact to the customer, determined by Customer Success and Support Engineering leadership
 - Failure to expedite scheduling may have cascading business impact to GitLab
- Approval from a VP from Customer Success AND a Director of Support Engineering are required to expedite scheduling
 - Customer Success: approval from VP, Customer Success Management - Sherrod Patching
 - Support Engineering: approval from VP, Support - Johnny Scarborough

Process

- The issue priority is set to ~"priority::1" regardless of severity
- The label ~"critical-customer-escalation" is applied to the issue
- The issue is scheduled within 1 business day
 - For issues of type features, approval from the Product DRI is needed.
- The DRI or their delegate provides daily process updates in the escalated customer slack channel

DRI

- If issue is type bug DRI is the Director of Development
- If issue is type feature DRI is the Director of Product
- If issue requires Infrastructure work the DRI is the Engineering Manager in Infrastructure

The DRI can use the customer critical merge requests process to expedite code review & merge.

Pairing Engineers on priority::1/severity::1 Issues

In most cases, a single engineer and maintainer review are adequate to handle a priority::1/severity::1 issue. However, some issues are highly difficult or complicated. Engineers should treat these issues with a high sense of urgency. For a complicated priority::1/severity::1 issue, multiple engineers should be assigned based on the level of complexity. The issue description should include the team member and their responsibilities.

Team Member	Responsibility
Team Member 1	Reproduce the Problem
Team Member 2	Audit Code Base for other places where this may occur

If we have cases where three or five or X people are needed, Engineering Managers should feel the freedom to execute on a plan quickly.

Following this procedure will:

- Decrease the time it takes to resolve priority::1/severity::1 issues
- Allow for a smooth handover of the issue in case of OOO or End of the Work Day
- Provide support for Engineers if they are stuck on a problem
- Provide another set of eyes on topics with high urgency or securing security-related fixes

Internal Engineering handbook

There are some engineering handbook topics that are internal only. These topics can be viewed by GitLab team members in the engineering section of the internal handbook.

SECTION: engineering/automation.md

Overview

Token management is crucial in providing authentication and authorization within various systems and subsystems used by GitLab. The token overview will assist in identifying the tokens used in GitLab. Refer to the Token Management Standard for approved token usage and distribution.

Guidelines for automation and access tokens

Automations for the gitlab-org group and projects under it can be split into three categories:

Automation for repository, packages, container registry within a project: a project deploy token is sufficient.

Automation via API within a project: a project access token is sufficient.

Automation via API within a group: a group access token is sufficient.

These guidelines ensure consistency for Engineering automation using approved secure patterns aligned with least privileged access principle.

Merge request automation guidelines

Automation that opens a merge request in projects under gitlab-org shall apply the ~"automation:bot-authored" label for clearer measurements for bot authored MRs that may cause an impact to Engineering PIs.

Access token best practices

Don't reuse an existing token

Never reuse an existing token for your own automation, as this makes it harder to track what a token is used for.

It also increases the number of changes required if the token is revoked.

Use project access tokens by default

By default, and when possible, create a new project access token for any API automation, and follow these guidelines:

Create a suitable name for the access token. Keep in mind that this is also the name of the bot user created for the token.

Always set an expiration date for the token, even if this is for temporary automation.

Give the token the minimum scopes your automation requires (usually the api scope).

Project access tokens have a few known limitations, but dogfooding them can only help us improve the feature.

Use a project deploy token for repository, packages, or container registry automation

Create a new project deploy token for repository, packages, or container registry automation.

Follow the same guidelines as for the project access tokens above.

Use a specific service account token when project access tokens can't be used

For cases where automation is applied at the group level or across multiple projects, and where the Maintainer

permission is required to perform the automation (for example, to post restricted quick actions), you can

request a new service account

that will be owned by a specific team.

List of automations currently in place

GitLab uses automation to streamline engineering processes, such as:

Danger bot for merge request hygiene. We use the Danger bot group access token.

Triage ops for automated:

- Scheduled reminders and reports of issues and merge requests. Requires a service account.

- Real-time reaction to events on issues and merge requests. Requires a service account.

- Allure test reports. We use the End-to-end tests Allure report project access token to post Allure test report on merge requests that run end-to-end tests against their Review App.

- Asynchronous retrospective generation. Can use a project access tokens unless fetching confidential issues.

- GitLab Runner releases. Requires a service account.

- Repository mirroring. Requires a service account.

Current and potential GitLab.com service accounts

@gitlab-bot owned by the Engineering Productivity team, running various things. We're breaking this down into multiple dedicated service accounts.

@gitlab-qa owned by the Developer Experience stage, running end-to-end test related automation.

@gitlab-release-tools-bot owned by the Delivery team, running delivery/release-related automation.

@gl-build-trigger owned by the Distribution group, triggering build-related pipelines.

@gitlab-omnibus-mirror-bot owned by the Distribution group, mirroring various dependency projects of the gitlab-org/omnibus-gitlab project.

Triage operations for gitlab-org/quality/triage-ops, gitlab-org/gitlab-triage owned by the Engineering Productivity team.

Background on the single @gitlab-bot service account

Previously, we had a single @gitlab-bot service account that we used for almost all of our automated processes.

This had two main drawbacks:

- If the bot hit the API rate limit, we had to disable the rate limit.

- This meant that an infinite-loop script could lead to a DoS (denial-of-service) against our own API.

- It caused security concerns:

 - Reusing the same token everywhere creates a lot of disruption if

the token is leaked and needs to be rotated.
The service account credentials were accessible to the whole Engineering division.
Anyone could log into the bot's account and create their own access token.

We are in the process of:

Moving the @gitlab-bot service account credentials to an Engineering Productivity 1Password vault, to stop the self-service of creating tokens.

Migrating the current @gitlab-bot's tokens to project access tokens where possible, and to specific service accounts for the other cases.

```
---  
## SECTION: engineering/cross-functional-prioritization.md  
  
{{% include "includes/cross-functional-prioritization.md" %}}
```

```
---  
## SECTION: engineering/cto-leadership-team.md
```

The CTO Leadership Team is composed of the CTO's direct reports and the Office of the CTO (OCTO).

Office of the CTO (OCTO)

The OCTO is composed of the CTO, the Engineering EBAs, the CTO's People Business Partners, and the CTO's Director of Technical Program Management. This team works to amplify the CTO's reach, vision, and mission. They work together to deliver programs and results across the entire Engineering Division.

Engineering All-Hands

The Engineering All-Hands meeting takes place twice per quarter, scheduled at the middle and end of each quarter, with time slots that accommodate both EMEA/AMER and APAC/AMER-friendly time zones. Twice per year, we offer alternative EMEA/APAC and AMER-friendly time slots to better support global team participation. Additionally, CTO Office Hours occur weekly, alternating between EMEA/AMER and APAC/AMER time zones each week.

The purpose is to share company updates, stay connected, and receive feedback.

The entire Engineering group is invited, though anyone at GitLab is welcome to attend and contribute to the All-Hands agenda (internal). As with all general meetings at GitLab, attendance is optional though encouraged, and will be recorded.

All Hands Prep Process

1. Two Weeks Before the All Hands:

1. CTO EBA will add a topic to the leadership weekly meeting to solicit discussion topics and assign DRIs.

2. CTO EBA will prepare a slide deck template and script doc for use by all presenters.
 3. Once topics and DRIs are confirmed, CTO EBA will create a temporary Slack channel named 'tmpYYYYMMengallhandspresenters' for presenters and stakeholders to discuss and align on content. This channel will be archived after the All Hands is completed.
 4. Stakeholders/Presenters are encouraged to invite additional stakeholders to the channel as needed or opt out if they do not plan to present.
2. Content Requirements and Guidelines:
1. Presenters should note the expected length of their presentation in the script doc to ensure proper time allocation.
 2. To ensure accessibility and consistency, speaker notes in the presentation deck should match the script doc. This eliminates the need for team members to provide real-time transcription during presentations.
 3. Presenters must include slide change indicators by manually highlighting the relevant text at the end of the sentence in the speaker notes and script doc.
 4. Stakeholders/Presenters must clearly indicate in the script doc under the Speaker column if different speakers are presenting in each session.
3. Content Development and Submission:
1. Presenters are accountable for completing their content following the above requirements, including speaker notes, and submitting it no later than 48 hours before the first session.
 2. Any updates or clarifications during development can be addressed in the Slack channel.
 3. CTO EBA will organize and share all final content in the agenda doc.
4. Day of All-Hands Meeting:
1. CTO EBA will schedule and facilitate a 15-minute tech check with all presenters for both sessions.
 2. Stakeholders should ensure additional presenters from their teams are included in the tech check if applicable.

CTO Leadership Offsite

The quarterly CTO Leadership Offsite provides a forum for longer-form discussion among the CTO's Leadership Team. This offsite is planned by the OCTO Team and rotates between in-person and virtual.

Schedule

The CTO Leadership Offsite schedule is currently being determined. We will update the handbook with the cadence once confirmed.

The offsite working days are scheduled from Tuesday to Thursday with travel days planned to accomodate arrivals in time for a Monday evening get together and departures on Thursday later afternoon.

Goals

Some of the goals for the offsite are documented below, but each offsite may have additional goals which would be covered.

1. Review and align on technical vision for the Engineering Division
1. DIB Overview & Goal Setting

1. Discussing cross-functional topics (e.g. communication, prioritization, etc.)
1. Reviewing organization design
1. Reviewing technical topics of interest

Agenda

The specific agenda for each offsite depends on active projects and the goals of the offsite. Typical items may include:

1. People
 1. Succession planning
 1. Did anything change in the succession plan compared to six months ago?
 1. Did anything change in the Talent Assessment for all VP's directs?
 1. Director+ promotion planning
 1. Hiring plan
 1. Relevant People projects around performance, engagement, and retention
1. Big rocks (large initiatives) presentation and discussion
1. Direction updates (annual in Q4)
1. Department-to-department feedback

In general, we prefer to have more items than we can fit into the agenda, and then push some items out, rather than have too little.

SECTION: engineering/demos.md

Occasionally, it is useful to set up a demo on a regular cadence to ensure cross-functional iterative alignment.

This is helpful for high-impact deliverables that require integration across multiple functional teams. This is in-line with the seventh principle of the Agile Manifesto: "Working Software is the best measure of progress".

Demo script

For multi-person groups or critical projects, we use a heavier weight grading process:

1. The demo owner identifies the outcome of the demo based on the business criteria. This can be an engineering manager, a product manager or someone who is a business stakeholder of the outcome.
1. The demo owner breaks down the outcome into smaller pieces, aligning with functional areas (tracks) and structured in procedural flow. This will later be captured as demo steps.
 - List each step, however small it might look to expose implicit dependencies.
1. The demo owner identifies a functional team leader as a DRI for each demo track. The DRI for each track is responsible for demoing each track to completion.
1. The demo owner collaborates with functional team leaders to populate the demo steps in a scorecard. Here is the demo scorecard template. To use this template:
 - Copy the template and rename to the initiative/deliverable.
 - Clear the scores in the scorecard sheet.
 - Populate the demo tracks and demo steps.

- Note: Here is an example of a populated demo scorecard.

1. The demo owner identifies a demo grader to hold grading accountability. This can be the demo owner or someone who is familiar with the product domain and customers' usecase. It is important that the demo grader is someone who can advocate for the success of our end users.

Demo scheduling

1. Once the script is finalized, the demo owner schedules a recurring recorded meeting for the demo with target end date.

1. Demo owner & demo grader must be present in every demo to ensure accountability. Assign delegates appropriately for one-off unavailability.

1. Create an agenda document where each participant can take notes in, in addition to the scorecard.

1. The audience is the key business stakeholder of the demo deliverables & the product group team (Development, UX, Quality, Product).

1. Meeting should be kept to 30 minutes. The emphasis should be on the product requirements & acceptance criteria.

1. The demo gets kicked off and each demo tracks iterate each week on the progress until completion.

1. Live streaming or uploading to GitLab Unfiltered channel is optional. Please abide by our SAFE guidelines if chosen to do so.

Demo grading

The demo master grades each step during the demo meeting. To make it less subjective, we use a scale that is widely understood and communicated.

Our scoring definitions are as follows:

| Score | Definition |

| ----- | ----- |

| 5 | Signed off from all Stakeholders |

| 4 | Demo'd according to the "definition of done" |

| 3 | Demo'd but incomplete and/or has bugs |

| 2 | Demo'd in some rudimentary state, work has started |

| 1 | Not yet started or cannot be demo'd |

Single Engineer Groups Demo

A different demo process is used by the Single-Engineer Groups and does not require a demo score card.

- Record a video on your computer or via Zoom livestream of the working software
- Upload to the GitLab Unfiltered Channel
- Post the YouTube link to the appropriate Slack channel
- Repeat periodically

SECTION: engineering/developer-onboarding.md

Awesome! You're about to become a GitLab developer!
Make sure you've checked out our handbook beforehand, so you get a feeling of how we work at GitLab. Below you'll find everything you need to start developing. If something is missing, add it (as goes with everything at GitLab)!

GitLab Environments

We have multiple GitLab environments.

On those instances, please enable the performance bar by pressing `p` then `b` (even on production.)

Then, read how to use and enable the production Canary on GitLab.com.

Getting started with GitLab development

To start development, follow the instructions for the GitLab Development Kit.

GitLab Repositories

GitLab consists of many subprojects. A curated list of GitLab Repositories can be found at the GitLab Engineering Projects page.

Almost all repositories are available on both GitLab.com and dev.gitlab.org. We also mirror to dev.gitlab.org for availability reasons and GitHub for historical reasons.

All issues should be filed on GitLab.com.

Infrastructure

For everything related to infrastructure, check out the Infrastructure handbook.

In particular production architecture might be useful for onboarding.

If you need a VPS for any reason, it's probably easiest to set one up at DigitalOcean. Ask another developer for access.

Basics of GitLab development

Workflow

Please see the engineering workflow document in the handbook and read the developer documentation.

Security

Read the developer security documentation prior to working on a security issue.

Quality

One of GitLab's strengths is its high quality of software. To achieve this we've introduced some requirements to all source code that is contributed. All requirements are mentioned in the Contribution guide.

Make sure you read and follow it.

Dependencies

GitLab can be installed through an Omnibus package or from source on different Linux distributions and macOS. In order to maintain portability, we need to avoid adding extra dependencies and use of exotic database extensions. Every time you choose between changes in the application code or adding new dependencies, you should give priority to the first because this is easier to maintain and set up. If you still need to bring new dependencies to GitLab, ask another developer or the CTO for advice.

Submit your code

In GitLab all code should go through a review process before it can be merged. Make sure you submit a merge request for any changes you've made. When the merge request is ready, it should be assigned to someone else on the team. This person is then responsible for reviewing your code and merging it. Optionally, you can request another developer to help or review by writing a comment. If a merge request is not assigned, it will probably be ignored and create unnecessary delays.

Don't work on one task for multiple days before submitting a merge request. Even the biggest task can be split into smaller tasks.

Try to submit a merge request for each part of the functionality.

This means that we expect multiple merge requests per week from you.

Smaller merge requests are more likely to receive good feedback and will get merged sooner.

Unless the change is very minor, or is fixing a bug that was introduced in the same version, create a changelog entry using the Changelog Git commit trailer.

Do not include your name in the entry as we only do that to give recognition to volunteer contributors.

Working on GitLab EE (developer licenses)

GitLab EE requires an active license key.

GitLab team members can follow these instructions to acquire or renew a license.

Wider community members should contact the Contributor Success team.

Ruby Gems

Follow the development guidelines for Ruby gems.

Get involved with the wider community

- Become a Merge Request Coach
- Join the Community Discord
- Encourage teams to host Community Office Hours
- Reach out to the Contributor Success team on Slack in contributor-success
- Involve the community in things you do
- Have conversations in public
- Triage team issues and label with quick win where applicable, especially in advance of Hackathons
- Use the Community Forks
- Consider live streaming and pairing sessions, with team members and/or the community
- Record meetings for the GitLab Unfiltered YouTube channel (handbook info here)
- Champion requests from community members and raise concerns internally
- Reach out offline to connect with community members on channels like X/Twitter or LinkedIn

Relevant links

- Engineering Handbook
- Engineering Workflow
- Product Handbook

SECTION: engineering/engineering-innovation.md

This guide serves as a comprehensive handbook for GitLab team members (engineers, product managers, and designers) who want to create innovations (proofs-of-concept) and share them internally and eventually, with the public in a transparent and collaborative way.

Alignment with GitLab's Strategy

Innovation at GitLab is important: It is a core component of our long-term strategic vision outlined in the Company Strategy. Our approach of "seed then nurture" ensures that early proofs-of-concepts can evolve into mature features that drive both user adoption and business growth.

This iterative and collaborative guide directly supports GitLab's three-year strategy by bridging short-term proofs-of-concepts with long-range planning. As innovations are refined and scaled, they reinforce our strategic priorities and allow us to adapt dynamically to evolving market trends.

Table of Contents

1. Overview

1. Internal vs External Innovations

1. Process: Creating and Managing an Innovation

1. Sharing Your Innovation

1. Additional Resources

Overview

Creating an innovation at GitLab means exploring new ideas and building prototypes to create new products or improve our existing products and processes. This guide outlines the steps you should follow, from ideation to sharing feedback, in both internal and external settings. An innovation can be a new feature, feature improvement, innovative use of GitLab feature to show unknown use cases, a new organizational tool, or anything that helps the success of GitLab.

- Innovation (Proof-of-Concept, or POC): A prototype created to validate a new idea or potential product feature.

- Epic Tracking: We use GitLab Epics to consolidate all tasks and issues related to the innovation under a single board (specifically the "Innovation at GitLab" board) for transparency and visibility.

> Note: Innovations are not yet Experiments - however, this guide will help your innovation become an experiment if applicable. When an innovation demonstrates success through monitoring metrics and positive feedback, it can be promoted to a full product feature through collaboration with our product organization.

Good to Know: The power of POCs lies in their simplicity and ability to quickly demonstrate value. Keep your POCs small and focused:

- Start with the minimum implementation needed to demonstrate the concept.
- Focus on proving technical feasibility and value proposition before full implementation. It's okay to simulate or stub certain components that are known to be technically feasible.
- Examples might include: a new prompt with example outputs, a basic UI mockup, or a simple dataset demonstration.

Remember: The goal is to validate ideas and demonstrate value quickly, not to build a complete feature.

Internal vs External Innovations

Innovations can be either internal or external. The goal for external innovations is to move your Proof-of-Concept to an experimental feature, while internal innovations are focused on improving GitLab's internal operations and team efficiency.

Here is a flowchart outlining the process:

mermaid
flowchart TD

A[What is my innovation/project?] --> B{Is it an external feature?}

B -->|Yes| C[Work with Product and UX]

B -->|No| D[Work with your engineering peers]

C --> E{Is the innovation accepted by Product and UX?}

E -->|Yes| F[Create an experiment]

E -->|No| G[Deprecate the innovation]

D --> J[Maintain as Internal]

D --> K[Upgrade to External / Experimental]

D --> I[Deprecate - not valuable]

K --> C

> Note: Consult your manager if you're unsure which path your innovation should take.

Internal Innovations

Internal innovations focus on developing tools, processes, and systems that improve GitLab's internal operations and team efficiency. These innovations:

- Are primarily designed for GitLab team members rather than external customers
- Focus on improving internal workflows, automation, or tooling
- May not require extensive Product or UX involvement in the early stages
- Follow the internal innovation process outlined in this guide

> Note: Internal innovations, or components of internal innovations, can sometimes evolve into external features when they demonstrate value that could benefit GitLab customers. In this case, DRIs should initiate collaboration with Product and UX teams and follow the external innovation process outlined in this guide.

External Innovations

External innovations are focused on developing new features or enhancements for the GitLab product that will directly impact customers. These innovations:

- Are intended to become part of the GitLab product offering
- Directly impact customer experience or user interface
- Should have early collaboration with Product and UX teams
- Should align with product roadmaps and strategic initiatives

> Important Note: All external innovations must be clearly labeled with "Proof-of-Concept" or similar.

Process: Creating and Managing an Innovation

Step 1: Create a Proposal and Get Approval of Your Manager

1. Create a concise abstract that includes:

1. Your core idea and vision
 1. The specific problem you're addressing
 1. The anticipated business value and impact with example use cases
 1. Key hypotheses you plan to validate in the timebox
 1. Timeboxed goals with clear targets
 1. Estimated time investment required
1. Be sure to consult your manager before investing time into an idea. Your manager may surface that this work is not well-aligned or is a lower priority compared to other initiatives. Formalize your pitch in a SPIKE issue and obtain explicit manager approval before investing significant time, then work with your manager to identify optimal timing for this timeboxed exploration
 1. Consider any cost implications with the innovation, such as infrastructure cost, license cost, and maintenance cost to the best of your knowledge.
 1. Additionally, work with your Manager on identifying any other subject-matter-experts that may be attempting to solve the same problem.

Step 2: Initiate an Epic

- Create a GitLab Epic: Start by creating an Epic on the "Innovation at GitLab" board. This will be the central hub for tracking all aspects of your innovation.

Step 3: Start work on the POC and create the Associated Issues

After the creation of the Epic you can in parallel start the actual work on the POC and the creation of the issues. Use the POC work to fill the information on the following sections.

Within the Epic, create the following issues:

1. Overview: This should be a high-level summary of the innovation detailing the purpose, expected business value, and a resource list for more information. This can also be in the Epic Description section. Your epic overview section should include the following:
 - Base Properties: all innovations should have the following:
 - A DRI (Directly Responsible Individual): for each innovation (and if moved to experimental, the DRI for the experiment). This person should be responsible for the cleanup of the innovation so that does not go stale.
 - Why Now? or Problem to Solve? - this should include the business case for the innovation. This should help everyone to understand the problem the POC is solving.
 - Dependencies or Implications: Any dependencies or implications for other product areas or teams.
 - Timeline: What will mark this innovation as "complete"?
 - Cost: What are the cost implications of this feature? Are there any infrastructure related costs? Expected maintenance cost?
 - External Innovation: If the innovation is intended for an external feature, include the following:
 - Stage and Group: Identify the stage and group this functionality would eventually be maintained and owned by - be sure to tag the Product Manager for that group.
 - Alignment with Product Roadmap and Strategic Initiatives: If possible, include a section on how this feature aligns with the Product Roadmap and Strategic Initiatives.
 - Security and Privacy: Security and Privacy considerations - has this been reviewed by the Security Team?

- Differentiation Plan: If applicable, describe how your innovation will be differentiated (visually, or otherwise) from standard product features, including:
 - Which visual differentiation approach you'll use (UI badge, experimental mode, feature flags, etc.).
 - How users will opt-in to see or use the innovation.
 - Any documentation or disclaimers that will accompany the innovation.

2. POC (Proof-of-Concept) or Demo: This issue should contain:

- A concise explanation (could be a text, a video, an image, etc.) of the Why of the idea, story behind the vision, potential business value and demonstrating the POC.
- Links to relevant repositories and code branches.
- Step-by-step instructions to set up the POC environment, if applicable.

3. Feedback: Document both internal feedback from GitLab team members and, when applicable, external user feedback.

4. Documentation (Optional): A detailed breakdown of the innovation's internals, including diagrams, code snippets, and technical explanations.

- This can serve as a "how it was made" guide.
- Since POCs are typically small and often involve stubbed parts, be sure to outline implementation dependencies.
- If possible, explain how your POC can be extended.

5. Monitoring (Optional): This issue should help users understand how your innovation is performing. You can use this issue to provide metrics and monitoring details for the innovation, such as links to dashboards or performance metrics. If monitoring data is confidential or sensitive, ensure the issue is handled accordingly.

Step 4: Consider Confidentiality

- Public by Default: In alignment with GitLab's Public by Default value, innovations should be public from the start if possible. This promotes:

- Enhanced trust and transparency with our community
- Potential community collaboration opportunities

- Review Confidentiality Guidelines: Before making your innovation public, review the Confidentiality Levels guidelines to determine if your innovation contains any sensitive information that should not be public, such as:

- Potentially patentable innovations (consult with Legal Team)
- Strategic initiatives that require temporary confidentiality

- Innovation Disclaimer: When sharing innovations publicly, include the following disclaimer at the top of your epic:

> This page contains information related to unofficial products, features, and functionality. It is important to note that the information presented is for informational purposes only. Please do not rely on this information for purchasing or planning purposes. The development,

release, and timing of any products, features, or functionality may be subject to change or delay and remain at the sole discretion of GitLab Inc.

- When in Doubt: If uncertain about the confidentiality requirements of your innovation, consult with your manager. It's better to start confidential and transition to public than to inadvertently disclose sensitive information.

> Note: If your innovation includes a video through the GitLab YouTube Unfiltered channel and your innovation is marked confidential, it's recommended to mark that video as private until the innovation is ready to be shared with the public.

Sharing Your Innovation

Start with sharing your innovation with a limited, curated group of users (for example, by enabling a feature flag for that group). Ask the group members to share their feedback in the Feedback issue that you created earlier in the Epic above.

Once you have gathered initial feedback and validated the potential of your innovation, these communication steps will help you share your innovation with the larger organization:

1. Get Internal Feedback:

- Get POC feedback from your Manager and Director+ in the Feedback issue created at step 3:
- If your innovation is external facing, collect input from Product and UX peers.
- If your innovation involves other product areas, lean into our short toes value by involving the correct teams early on.

2. Share Your Innovation!

- Post your innovation summary in the \innovation channel to share it with the larger organization and begin cross-team conversations. Share it with any other relevant channel as well.
- Tag anyone relevant in your Epic's issue description.

3. Cross-Team Collaboration:

- When your innovation extends beyond your team's domain, partner with relevant counterparts (engineering, UX, and product) to refine and handover the idea effectively.

4. If Confidential, Transition The Epic to Public:

- In line with GitLab's Public by Default value, work with the corresponding Product and UX team to make your innovation public as soon as possible once you've validated its potential. If your innovation contains potentially patentable information, work with your manager and the Legal Team to see if it meets patent requirements.
- Ensure the transition aligns with GitLab's Company Strategy and Values, and collaborate

with Product and UX when publicly communicating the innovation.

Completing the Innovation process

Once the work on your innovation is done, it is important to close out the associated issues and epics and come to a decision on the path forward. This ensures that the GitLab issue tracker remains clean and easy to navigate.

An innovation can be closed out in a few ways:

- The innovation work came to a conclusion to not move forward
- The innovation is ready that a decision can be made to either be promoted to an product feature experiment, will be implemented (technical innovations) or for internal innovation will be kept longterm internally
 - For making this decision ping the PM or EM for the specific area and ask for making a clear decision and the Why behind the decision
- There are legal or security concerns that prevent the innovation from moving forward.

Additional Resources

- GitLab Engineering Handbook: Refer to the GitLab Engineering Handbook for further context on engineering best practices and standards.
- GitLab Values: Understand more about our core principles in the GitLab Values page.
- Company Strategy: Learn how innovations fit within our larger Company Strategy.
- Architecture Design Workflow: Architectural innovation usually follows our Architecture Design Workflow

This guide is intended to be a living document. As GitLab evolves and our approach to innovation grows, please update this guide with any new processes or recommendations.

SECTION: engineering/expansion-development.md

Vision

Scale and develop our diverse, global team to drive results that support our product and customer growth, while maintaining our values and unique way of working.

Mission

GitLab's unique way of working asynchronously, handbook first, using the product we develop, and with clear focus on our values enables very high productivity. In delivering on growth, we maintain our values and ways of working while developing team members and increasing the diversity of our team. We focus on constantly improving usability and reliability of our

product to reach maximum customer satisfaction. Community contributions and customer interactions rely on efficient and effective communication. We are a data-driven, customer experience first, open core organization delivering one secure, reliable, world leading DevOps platform.

Efficiency

GitLab's Development group ships thousands of product merge requests per month. Continuing to scale our development process to an ever larger number of contributors requires efficiency, collaboration, and iteration. In FY24, we want to keep our MR Rate stable as we continue to onboard new team members, increasing